

Name	AYUSHI JAPSARE
UID no.	2023301006
EXP	5

CSS	
AIM:	To implement hash functions (MD5, SHA-1, SHA-256, SHA-512) in Python, generate checksum files for verifying file integrity, and detect tampering
PROBLEM STATEMENT:	A hash function is a one-way mathematical function that produces a fixed-length digest from any input. Properties of cryptographic hash functions: 1. Deterministic – Same input always produces the same hash. 2. Collision-resistant – Hard to find two inputs with the same hash. 3. Avalanche effect – Small change in input → large change in output Checksum files contain the pre-computed hash of a file and can be used to verify integrity after download/transfer. MD5 and SHA-1 are outdated (vulnerable to collision attacks). SHA-256/SHA-512 are secure and widely used in modern systems (e.g., verifying Linux distributions, software packages). This experiment demonstrates the role of cryptographic hash functions in ensuring data integrity. ● First, the experiment covers the generation of different hash values (MD5, SHA-1, SHA-256, SHA-512) of a file. ● Second, the experiment focuses on creating checksum files to store hashes for later verification. ● Finally, checksum verification is performed to detect whether a file has been modified or tampered during transmission or storage.

THEORY

1. Introduction

In computer security and data communication, hash functions and checksum mechanisms play a crucial role in ensuring data integrity. When files are transmitted over networks or stored for long periods, they can be corrupted accidentally (due to hardware/software errors) or tampered with intentionally (due to malicious activity).

By computing and comparing cryptographic hash values, we can verify the authenticity of a file.

2. Hash Functions

A **hash function** is a one-way mathematical function that takes an input (file/message) of arbitrary length and produces a fixed-length **hash value** (digest).

Properties of Cryptographic Hash Functions

1. **Deterministic** → Same input always gives the same hash.
2. **Collision Resistance** → Hard to find two different inputs with the same hash.
3. **Preimage Resistance** → Hard to reconstruct the original input from the hash.
4. **Avalanche Effect** → A tiny change in input produces a drastically different hash output.

Common Hash Algorithms Used

- **MD5 (128-bit)**: Fast but **insecure**; vulnerable to collisions.
- **SHA-1 (160-bit)**: More secure than MD5 but also deprecated due to collision attacks.
- **SHA-256 (256-bit)**: Strong security; widely used for digital signatures, blockchain, and software verification.
- **SHA-512 (512-bit)**: Stronger than SHA-256; used when higher security margins are required

3. Checksums

A checksum file contains a pre-computed hash value of a file, stored separately. When a file is received or accessed later, its hash is recomputed and compared with the checksum file:

- If they match → File Authentic (Untampered).
- If they differ → File Corrupted/Tampered.

This technique is widely used in Linux distributions (ISO verification), software downloads, package managers, and blockchain transactions.

4. Workflow of the Experiment

1. Hash Generation

- Compute MD5, SHA-1, SHA-256, SHA-512 hashes for a given file.
- Store them in `hash_report.txt`.

2. Checksum File Creation

- Create `filename.sha256` containing the SHA-256 hash and filename.

3. Checksum Verification

- Recompute file's SHA-256 hash.
- Compare with stored checksum.
- Display result:
 -  Checksum OK (Authentic)
 -  Checksum FAILED (Tampered)

4. Tampering Test

- Modify the file (add/remove character).

- Re-run verification.
- Observe checksum failure (avalanche effect).

5. Applications of Hashing and Checksums

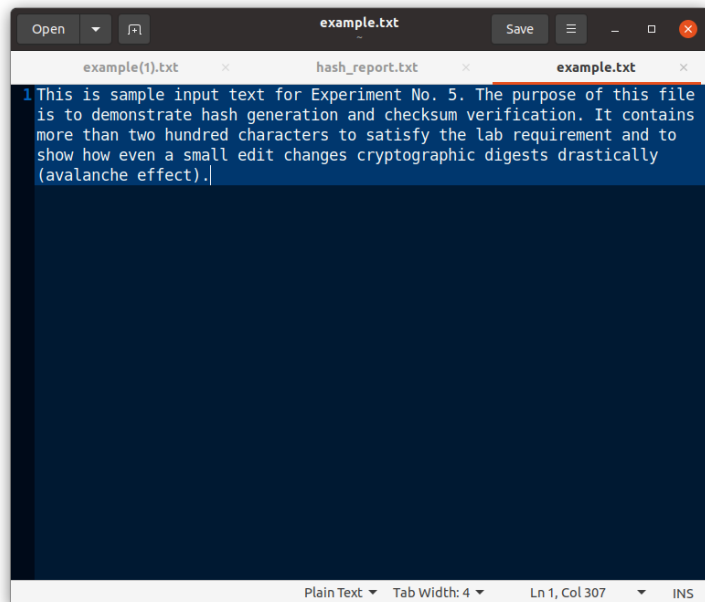
- **Data Integrity Verification** (downloads, backups).
- **Password Storage** (hashed passwords in databases).
- **Blockchain & Cryptocurrencies** (Bitcoin uses SHA-256).
- **Digital Signatures & Certificates.**
- **Intrusion Detection Systems (IDS)** (detect file tampering).

COLAB:

 [cssexp5.ipynb](#)

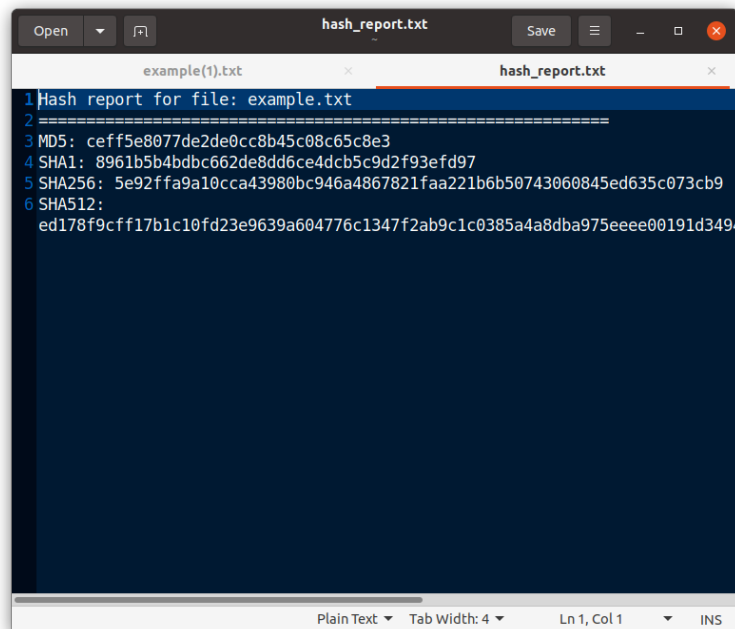
OUTPUT:

Sample file example.txt:



```
1 This is sample input text for Experiment No. 5. The purpose of this file
is to demonstrate hash generation and checksum verification. It contains
more than two hundred characters to satisfy the lab requirement and to
show how even a small edit changes cryptographic digests drastically
(avalanche effect).|
```

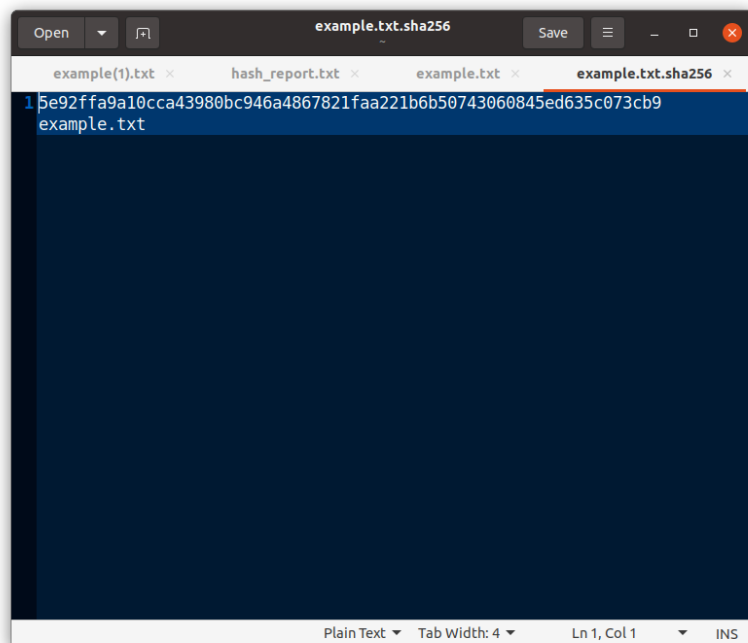
Generated Hash Report and checksum:



The screenshot shows a text editor window with the title 'hash_report.txt'. The editor contains the following text:

```
1 Hash report for file: example.txt
2 =====
3 MD5: ceff5e8077de2de0cc8b45c08c65c8e3
4 SHA1: 8961b5b4bdbc662de8dd6ce4dcb5c9d2f93efd97
5 SHA256: 5e92ffa9a10cca43980bc946a4867821faa221b6b50743060845ed635c073cb9
6 SHA512:
   ed178f9cff17b1c10fd23e9639a604776c1347f2ab9c1c0385a4a8dba975eeee00191d3494
```

The status bar at the bottom indicates 'Plain Text', 'Tab Width: 4', 'Ln 1, Col 1', and 'INS'.

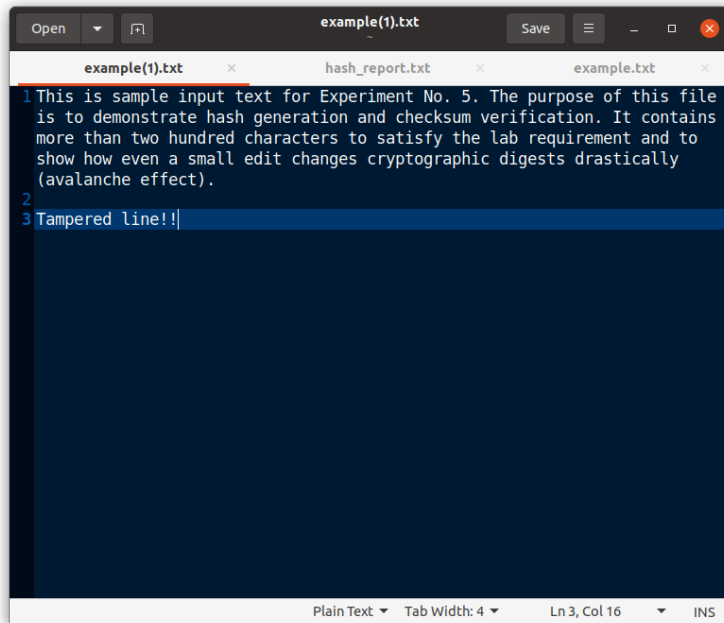


The screenshot shows a text editor window with the title 'example.txt.sha256'. The editor contains the following text:

```
1 5e92ffa9a10cca43980bc946a4867821faa221b6b50743060845ed635c073cb9
   example.txt
```

The status bar at the bottom indicates 'Plain Text', 'Tab Width: 4', 'Ln 1, Col 1', and 'INS'.

Tampered file:



```
[6] ✓ 0s verify_checksum(file_path, Path("example.txt.sha256"))
Checksum OK (Authentic)

[7] ✓ 0s tamper_file(file_path)
verify_checksum(file_path, Path("example.txt.sha256"))

[!] File tampered
Checksum FAILED (Tampered)
Expected: 5e92ffa9a10cca43980bc946a4867821faa221b6b50743060845ed635c073cb9
Actual:   b96f3af524255fdb66d13028faf539f280efb18021629a28109aef8f31b76dd2
```

Conclusion:

This experiment demonstrates the use of cryptographic hash functions and checksum files for verifying data integrity. A sample text file was taken, and its hash values were generated using MD5, SHA-1, SHA-256, and SHA-512, which were recorded in a hash report. A checksum file containing the SHA-256 hash was then created to enable later verification. During verification, the file's hash was recomputed and compared with the checksum to confirm authenticity. A tampering test was also performed by modifying the file slightly, which resulted in a mismatch and detection of tampering due to the avalanche effect. The experiment highlights the importance of secure hash functions, showing that while MD5 and SHA-1 are outdated, SHA-256 and SHA-512 are reliable for ensuring file authenticity in real-world applications like software verification and data security.